CSCI 3302 Introduction to Robotics

Lab 6: Gradient Descent Inverse Kinematics for Pick and Place

Due Date	**November 12th @ 11:59 PM**
Submission	**Canvas, lab6.py + report**
Points	**100 + 10 extra credit points**

Overview

In this lab, we will go over gradient descent inverse kinematics to solve a pick and place task. We will use a UR5e robot arm to pick up a red block and place it in the green bin. You will implement forward kinematics, a numerical Jacobian, and gradient descent inverse kinematics to compute the joint angles needed to reach target positions in 3D space.

The robot uses a Robotiq 3F gripper for grasping. The gripper interface is provided. Your job is to implement kinematics, math and task sequencing.

Background

Forward Kinematics and the Denavit-Hartenberg Parameters

Forward kinematics (FK) computes the end-effector position and orientation given a set of joint angles. For a manipulator with *n* joints, FK chains together *n* homogeneous transformation matrices, one per joint, to determine where the gripper ends up in 3D space.

The key to computing these transforms systematically is the **Denavit-Hartenberg (DH) convention**, introduced by Hartenberg and Denavit in 1955. DH provides a standard "recipe" for assigning coordinate frames to each joint of a manipulator and expressing the transform between consecutive joints using exactly four parameters. This is covered in detail in *Chapter 3.1.2 of the course textbook* (Nikolaus Correll, Bradley Hayes, Christoffer Heckman, and Alessandro Roncone, *Introduction to Autonomous Robots: Mechanisms, Sensors, Actuators, and Algorithms*).

The four DH parameters for each joint are:

- r (or a), link length: The distance between the z-axes of consecutive joints, measured along the common normal (x-axis).
- α (alpha), link twist: The angle between the z-axes of consecutive joints, measured about the common normal.
- d, the link offset: The distance between the x-axes of consecutive joints, measured along the z-axis.
- θ (theta), joint angle: The angle between the x-axes of consecutive joints, measured about the z-axis. For revolute joints (like the UR5e), this is the variable controlled by each motor.

These four parameters are combined into a single 4×4 homogeneous transformation matrix (see Equation 3.9 in the textbook):

$$
{}^{n}_{n-1}T = T'_z(d_n)R'_z(\theta_n)T_x(r_n)R_x(\alpha_n)
$$

which, when multiplied out, gives:

$$
{}^{n}_{n-1}T = \left[\begin{array}{ccc|c}
\cos\theta_n & -\sin\theta_n\cos\alpha_n & \sin\theta_n\sin\alpha_n & r_n\cos\theta_n \\
\sin\theta_n & \cos\theta_n\cos\alpha_n & -\cos\theta_n\sin\alpha_n & r_n\sin\theta_n \\
0 & \sin\alpha_n & \cos\alpha_n & d_n \\
\hline
0 & 0 & 0 & 1
\end{array}\right]
$$

The good news: you do not need to fully understand the derivation of the DH convention to *use* it. The `dh(a, d, alpha, theta)` function is provided for you and implements this matrix exactly. Your task is to chain together 6 of these matrices (one per UR5e joint) and extract the final position. The DH parameters for the UR5e are also provided as constants in the template.

The Jacobian

The Jacobian is a matrix that relates small changes in joint angles to small changes in end-effector position. For position-only control (which is what we need for pick-and-place), the Jacobian is a 3×6 matrix: 3 outputs (x, y, z velocity) and 6 inputs (joint velocities).

The key insight is this: the Jacobian allows us to compute inverse kinematics **numerically** rather than **analytically**. Analytical IK requires deriving closed-form equations that map desired positions to joint angles; this is extremely difficult for a 6-DOF arm. Instead, we can build our Jacobian directly from our forward kinematics (Task 1) by simply measuring how each joint affects the end-effector position. This is the finite difference approach: wiggle each joint by a tiny amount, see how the FK output changes, and record that as a column of the Jacobian.

Once we have the Jacobian, we can use it in a gradient descent loop (Task 4) to iteratively find joint angles that reach any target position, without ever solving a closed-form equation.

You will compute the Jacobian numerically using central finite differences:

$$J[:, i] = (FK(q + \delta e_i) - FK(q - \delta e_i)) / (2\delta)$$

where e_i is the unit vector in the direction of joint i and δ is a small perturbation (1e-5).

Gradient Descent Inverse Kinematics

Inverse kinematics (IK) finds joint angles that place the end-effector at a desired position. We use the Jacobian Transpose method, which is a form of gradient descent in joint space:

$$q \leftarrow q + \alpha \cdot J^T \cdot (target - FK(q))$$

Here, $J^T \cdot$ error gives the direction in joint space that reduces the position error most steeply. The learning rate α controls the step size. We iterate until the position error is below a tolerance (5mm). This approach is robust and always well-defined—unlike computing J^{-1} which can fail at singularities.

Task 1: Forward Kinematics (15 pts)

Implement forward_kinematics(q) which takes 6 joint angles and returns the gripper tip [x, y, z] position in the robot base frame.

- Chain the 6 DH transforms using the provided _dh() function and UR5e DH parameters.
- Apply the tool offset (GRIPPER_OFFSET = 0.18m) as a translation along the local Z-axis of the final frame.
- Return the XYZ position: the first 3 elements of the last column of the resulting 4×4 matrix.

Hint: Start with T = np.eye(4), loop over joints doing T = T @ _dh(...), then apply the gripper offset.

Task 2: Numerical Jacobian (15 pts)

Implement compute_jacobian(q) which returns the 3×6 position Jacobian computed via central finite differences. For each of the 6 joints, perturb q[i] by ±δ, evaluate your FK, and compute the partial derivative. This directly uses your FK from Task 1.

Task 3: Pick-and-Place Waypoints (10 pts)

Implement get_waypoints(block_world, bin_world) which returns a dictionary of 3D target positions (in world frame) that the gripper should visit during the pick-and-place. Think about: what approach height avoids collisions? Where exactly should the gripper be when it grasps? How high should it lift before moving laterally?

Task 4: Gradient Descent IK (20 pts)

Implement gradient_descent_ik() using the Jacobian Transpose method. The algorithm is:

1. Convert the world-frame target to the base frame using world_to_base().
2. Compute the position error: error = target_base − forward_kinematics(q).
3. Compute the Jacobian: J = compute_jacobian(q).
4. Update joints: q = q + learning_rate · J^T @ error.
5. Clip joints to [−2π, 2π] and repeat until ||error|| < tolerance.

Task 5: Pick-and-Place Sequence (10 pts)

Implement `pick_and_place()` to execute the full task. Use your IK to compute joint angles for each waypoint from Task 3, then command the arm through the sequence with appropriate gripper commands.

Report (30 pts)

Write a 2-3-page report addressing:

1. **Verification (9 pts):** Test your FK at 3 different joint configurations. For each, report the joint angles, the FK output, and the corresponding world position. Verify at least one by visually checking in Webots. Explain what GRIPPER_OFFSET does and why it matters.
2. **IK Analysis (9 pts):** Run your IK on 3 different target positions. For each, report: iterations to converge, final error, and joint angles found. Experiment with learning rate:

what happens at 0.1 vs 0.5 vs 2.0? Plot error vs. iteration for one target at different learning rates.

3. **Reflection (9 pts):** What is the role of the Jacobian in the IK algorithm? Why do we use J^T instead of J^{-1}? What limitations does this approach have? (Hint: think about orientation, singularities, and local minima.) Include a screenshot of the completed pick-and-place.

4. **Team Contributions (3 pts):** What did each member of the team contribute to completing the lab?

API Reference

Method	Description
arm.move_to(q, wait_time)	Move to joint angles, wait to arrive
arm.open_gripper()	Open the Robotiq 3F gripper
arm.close_gripper()	Close gripper (fingertip pinch)
arm.wait(seconds)	Wait for simulation time
arm.set_speed(speed)	Set joint velocity (rad/s)
_dh(a, d, alpha, theta)	Provided: 4×4 DH transform matrix
world_to_base(pos) / base_to_world(pos)	Provided: convert between world and base frames
forward_kinematics(q)	YOUR IMPL: gripper [x,y,z] in base frame
compute_jacobian(q)	YOUR IMPL: 3×6 position Jacobian
gradient_descent_ik(target, q0)	YOUR IMPL: solve IK via J^T method

Extra credit (10 pts)

Given the new world file (lab_6_extra.wbt), with four blocks of different colors, (gray, red, blue, green) use the same IK solver to stack the blocks on top of one another. The color sequence should be red -> blue -> green -> gray (first).

Note: The task should be a simple variable to switch between pick and place and block stacking. `Task = "block_stack" or "pick_and_place"`